Table of Contents

Introduction **2**

Views on the Expanded Database **2**

Create OrderItem **8**

Dropdown List **12**

Demo: Shop database, expanded CRUD

Author: Baifan Zhou
Detailed explanations are created with the help of ChatGPT and Gemini.

Introduction

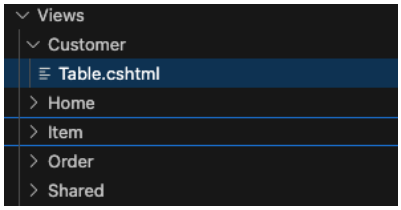
This demo covers the implementation of new views, the creation of order items, and the integration of dropdown lists for improved user input.

- **Add and Configure Views for Customers and Orders**: Begin by creating Razor Views of Table to present data for `Customer` and `Order` entities.
- **Create Controllers**: Implement controllers for `Customer` and `Order` to manage the data retrieval and interactions for these entities. We will update the controllers to support asynchronous operations and integrate them with our views.
- **Implement Advanced CRUD Operations**: Develop a feature for creating new `OrderItem` records. This includes adding a button to initiate the creation process, building a Razor view for the creation form, and coding the controller action methods to handle user input and validate data.
- **Enhance User Experience with Dropdown Lists**: Improve form usability by replacing text inputs with dropdown lists for selecting existing items and orders.

By following these steps, you will enhance your web application's ability to handle complex data interactions and provide a more user-friendly interface for managing and creating data entries. Happy coding!

Views on the Expanded Database

We have a lot of data, but they are not shown in the web application.
We now add some Views to display and change them.
First add the folders "Customer", "Order", under Views.
Add a Razor View Table.cshtml under Views/Customer



Modify the Customer/Table.cshtml to this:

```
Table.cshtml
1   @model IEnumerable<Customer>
2
3   <h1>List of Customers</h1>
4
5   <partial name="_CustomerTable" model="Model" />
6
```

Add a partial view (Razor View) _CustomerTable.cshtml under Views/Shared/

_CustomerTable.cshtml

```
@model IEnumerable<Customer>

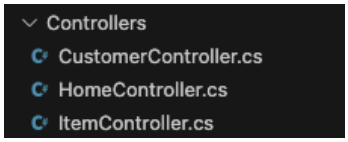
<div class="container">
    <table class='table table-striped'>
        <tr>
            <th>Id</th>
            <th>Name</th>
            <th>Address</th>
            <th>Orders</th>
            <th>Actions</th>
        </tr>
        @foreach (var customer in Model)
        {
            <tr>
                <td>@customer.CustomerId</td>
                <td>@customer.Name</td>
                <td>@customer.Address</td>
                <td>
                    @if (@customer.Orders != null)
                    {
                        <p>@string.Join(", ", @customer.Orders.Select(order => $"Order{order.OrderId}"))</p>
                    }
                </td>
                <td></td>
            </tr>
        }
    </table>
</div>
```

Modify the Views/Shared/_Layout.csthml (Line28 – 30) to allow access to Customer View

_Layout.cshtml

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>@ViewData["Title"] - My Shop</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/bootstrap.min.css" rel="stylesheet">
    <script src="~/lib/bootstrap/js/bootstrap.js"></script>
    <script src="~/lib/jquery/jquery.js"></script>
</head>
<body>
    <header>
        <nav class="navbar navbar-expand-sm navbar-toggleable-sm navbar-light bg-white border-bottom box-shadow mb-3">
            <div class="container-fluid">
                <a class="navbar-brand" asp-area="" asp-controller="Home" asp-action="Index">My Shop</a>
                <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target=".navbar-collapse" aria-
                  controls="navbarSupportedContent"
                        aria-expanded="false" aria-label="Toggle navigation">
                    <span class="navbar-toggler-icon"></span>
                </button>
                <div class="navbar-collapse collapse d-sm-inline-flex justify-content-between">
                    <ul class="navbar-nav flex-grow-1">
                        <li class="nav-item">
                            <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-action="Index">Home</a>
                        </li>
                        <li class="nav-item">
                            <a class="nav-link text-dark" asp-area="" asp-controller="Item" asp-action="Table">Items</a>
                        </li>
                        <li class="nav-item">
                            <a class="nav-link text-dark" asp-area="" asp-controller="Customer" asp-action="Table">Customers</a>
                        </li>
                    </ul>
                </div>
            </div>
        </nav>
    </header>
    <div class="container">
        <main role="main" class="pb-3">
            @RenderBody()
        </main>
    </div>
</body>
</html>
```

Add a CustomerController.cs under Controllers

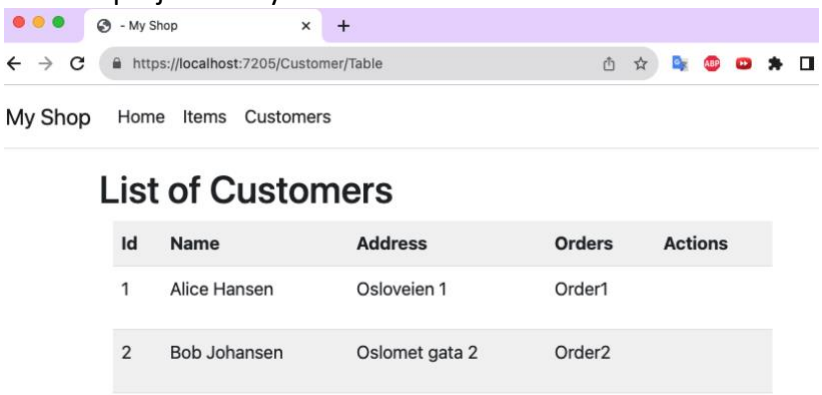


Change the CustomerController.cs to allow access to the Customer Table (async):

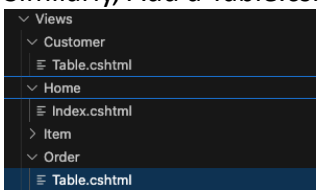
CustomerController.cs

```
election
1   using Microsoft.AspNetCore.Mvc;
2   using MyShop.Models;
3   using Microsoft.EntityFrameworkCore;
4
5   namespace MyShop.Controllers;
6
7   public class CustomerController : Controller
8   {
9       private readonly ItemDbContext _itemDbContext;
10
11      public CustomerController(ItemDbContext itemDbContext)
12      {
13          _itemDbContext = itemDbContext;
14      }
15
16      public async Task<IActionResult> Table()
17      {
18          List<Customer> customers = await _itemDbContext.Customers.ToListAsync();
19          return View(customers);
20      }
21  }
```

Run the project and you would be able to see the customer table:



Similarly, Add a Table.cshtml Razor View under Views/Order



Modify its code (The style attribute in Line7 adds a box that contains the order table and order item table):

Table.cshtml

```
1   @model IEnumerable<Order>
2
3   <h1>List of Orders</h1>
4
5   @foreach (var order in Model)
6   {
7       <div style="border: 1px solid #ccc; padding: 10px; margin-bottom: 20px;">
8           <div class="container">
9               <h5>Order @order.OrderId </h5>
10              <div>
11                  <table class='table table-striped' style='max-width: 1200px;'>
12                      <tr class="table-header">
13                          <th>Order Date</th>
14                          <th>CustomerId</th>
15                          <th>Customer Name</th>
16                          <th>Address</th>
17                      </tr>
18                      <tr>
19                          <td>@order.OrderDate</td>
20                          <td>@order.CustomerId</td>
21                          <td>@order.Customer.Name</td>
22                          <td>@order.Customer.Address</td>
23                      </tr>
24                  </table>
25              </div>
26          </div>
27          <partial name="_OrderItemTable" model="order.OrderItems" />
28      </div>
29  }
```

Add a partial view _OrderItemTable.cshtml under Views/Shared and modify its code:

_OrderItemTable.cshtml

```
1   @model IEnumerable<OrderItem>
2
3   <div class="container">
4       <table class='table table-striped'  style='max-width: 1200px;'>
5           <tr>
6               <th>OrderItemId</th>
7               <th>ItemId</th>
8               <th>Item</th>
9               <th>Quantity</th>
10              <th>OrderItemPrice</th>
11              <th>Actions</th>
12          </tr>
13          @foreach (var orderItem in Model)
14          {
15              <tr>
16                  <td>@orderItem.OrderItemId</td>
17                  <td>@orderItem.ItemId</td>
18                  <td>@orderItem.Item.Name</td>
19                  <td>@orderItem.Quantity</td>
20                  <td>@orderItem.OrderItemPrice</td>
21                  <td></td>
22              </tr>
23          }
24      </table>
25  </div>
```

Add a OrderController.cs under Controllers, and modify its code:

OrderController.cs

selection

```
using Microsoft.AspNetCore.Mvc;
using MyShop.Models;
using Microsoft.EntityFrameworkCore;

namespace MyShop.Controllers;

public class OrderController : Controller
{
    private readonly ItemDbContext _itemDbContext;

    public OrderController(ItemDbContext itemDbContext)
    {
        _itemDbContext = itemDbContext;
    }

    public async Task<IActionResult> Table()
    {
        List<Order> orders = await _itemDbContext.Orders.ToListAsync();
        return View(orders);
    }
}
```

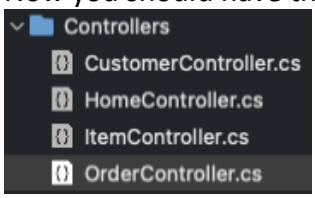
Modify the _Layout.cshtml to allow access to Orders (Line31 – Line33)

_Layout.cshtml

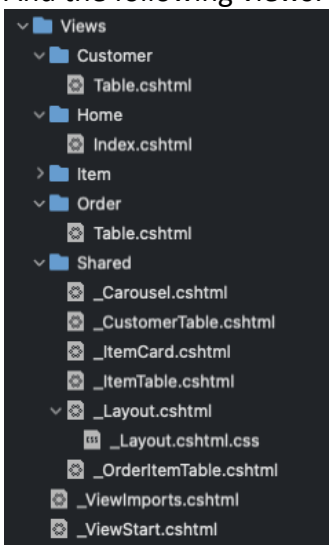
Views > Shared > _Layout.cshtml

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>@ViewData["Title"] - My Shop</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/bootstrap.min.css" rel="stylesheet">
    <script src="~/js/bootstrap.js"></script>
    <script src="~/js/jquery.min.js"></script>
</head>
<body>
    <header>
        <nav class="navbar navbar-expand-sm navbar-toggleable-sm navbar-light bg-white border-bottom box-shadow mb-3">
            <div class="container-fluid">
                <a class="navbar-brand" asp-area="" asp-controller="Home" asp-action="Index">My Shop</a>
                <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target=".navbar-collapse"
                aria-controls="navbarSupportedContent"
                        aria-expanded="false" aria-label="Toggle navigation">
                    <span class="navbar-toggler-icon"></span>
                </button>
                <div class="navbar-collapse collapse d-sm-inline-flex justify-content-between">
                    <ul class="navbar-nav flex-grow-1">
                        <li class="nav-item">
                            <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-action="Index">Home</a>
                        </li>
                        <li class="nav-item">
                            <a class="nav-link text-dark" asp-area="" asp-controller="Item" asp-action="Table">Items</a>
                        </li>
                        <li class="nav-item">
                            <a class="nav-link text-dark" asp-area="" asp-controller="Customer" asp-action="Table">Customers</a>
                        </li>
                        <li class="nav-item">
                            <a class="nav-link text-dark" asp-area="" asp-controller="Order" asp-action="Table">Orders</a>
                        </li>
                    </ul>
                </div>
            </div>
        </nav>
    </header>
    <div class="container">
        <main role="main" class="pb-3">
            @RenderBody()
        </main>
    </div>
</body>
</html>
```

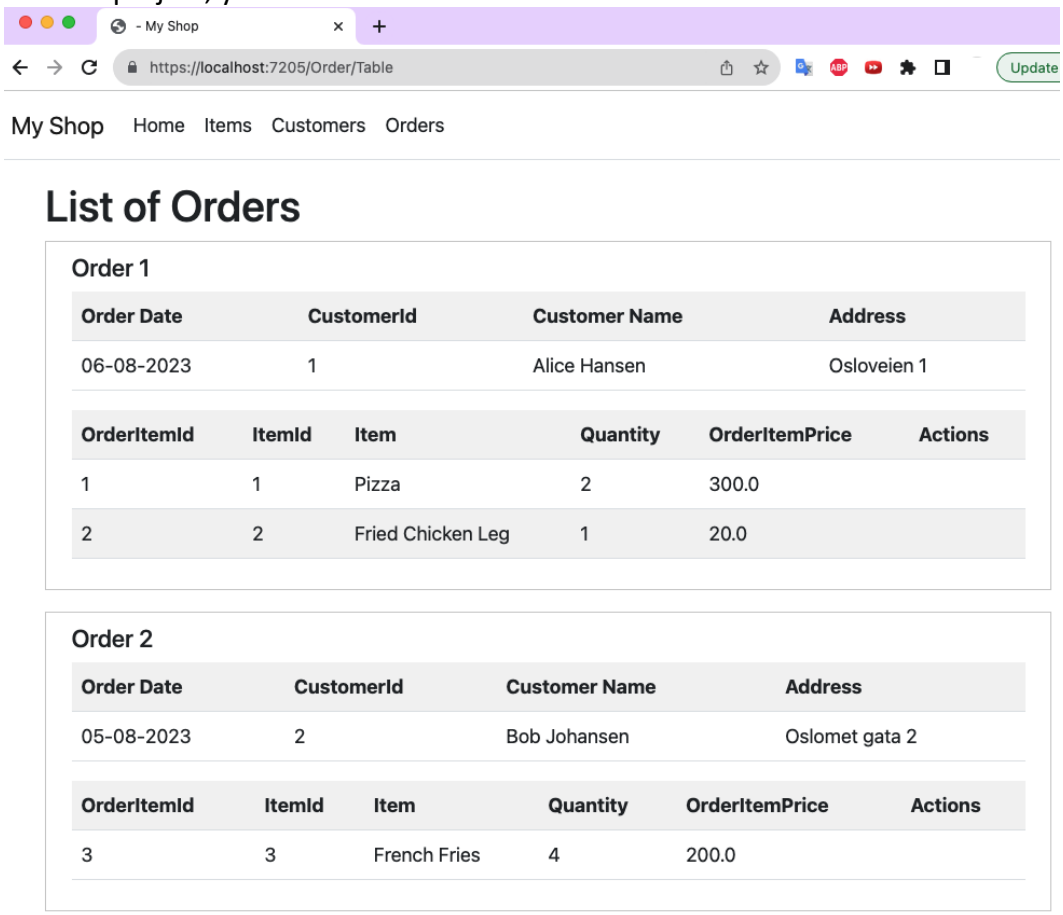
Now you should have the following controllers:



And the following views:



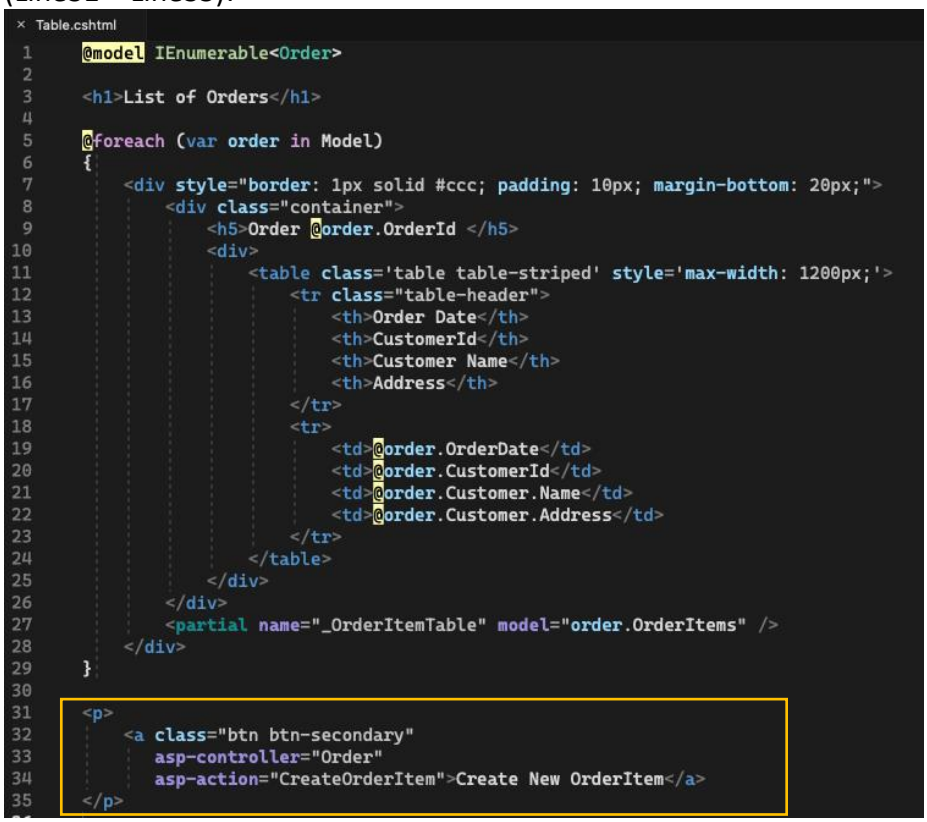
Run the project, you should be able to see the Orders:



Create OrderItem

Now we show an example of somewhat advanced CRUD. We introduce the creation of new OrderItem, which depends on Order.

We first create a new button under the Order Tables by modifying Views/Order/Table.csthml (Line31 – Line35):



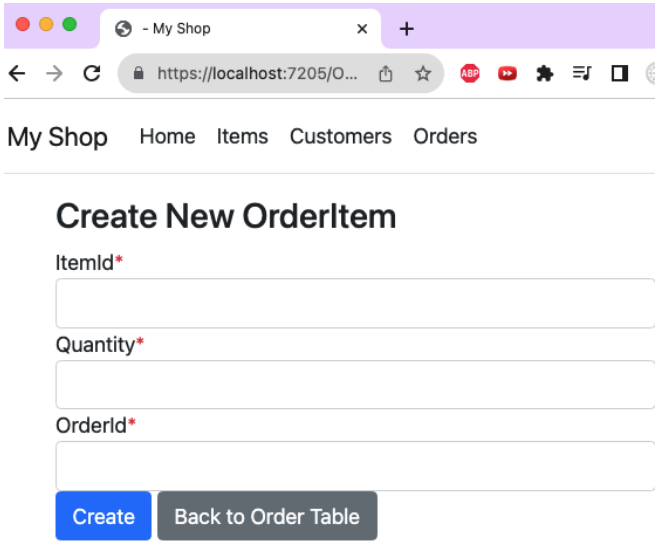
Then we add a View for the CreateOrderItem, we create a new Razor View named as CreateOrderItem.cshtml under Views/Order/

Copy the code from Views/Item/Create.csthml and modify the CreateOrderItem.cshtml as follows:
Line1: change model to OrderItem
Line3: change the title
Line5: change the asp-action
Line6 – Line20, change the asp-for attribute for the model bindings of OrderItem
Also note that all three forms groups are mandatory, so we put the red asterisk for all three.
Line22: change the back button

CreateOrderItem.cshtml

```
1   @model OrderItem
2
3   <h2>Create New OrderItem</h2>
4
5   <form asp-action="CreateOrderItem">
6       <div class="form-group">
7           <label asp-for="ItemId"></label><span class="text-danger">*</span>
8           <input asp-for="ItemId" class="form-control" />
9           <span asp-validation-for="ItemId" class="text-danger"></span>
10      </div>
11      <div class="form-group">
12          <label asp-for="Quantity"></label><span class="text-danger">*</span>
13          <input asp-for="Quantity" class="form-control" />
14          <span asp-validation-for="Quantity" class="text-danger"></span>
15      </div>
16      <div class="form-group">
17          <label asp-for="OrderId"></label><span class="text-danger">*</span>
18          <input asp-for="OrderId" class="form-control" />
19          <span asp-validation-for="OrderId" class="text-danger"></span>
20      </div>
21      <button type="submit" class="btn btn-primary">Create</button>
22      <a asp-action="Table" class="btn btn-secondary">Back to Order Table</a>
23  </form>
```

The defined CreateOrderItem view will look like this:



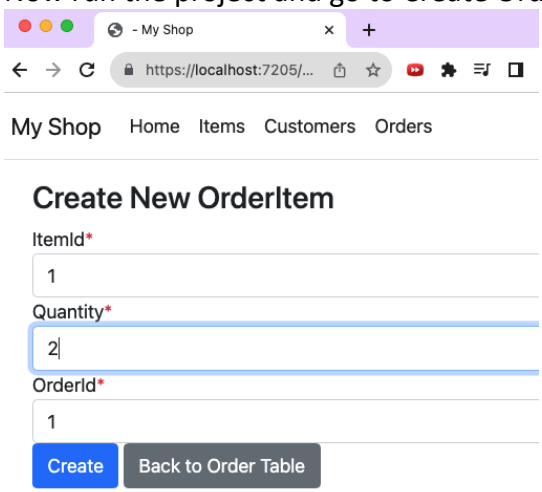
Now we go to the Controllers/OrderController.cs to create the action methods for CreateOrderItem:
First create the [HttpGet]CreateOrderItem() (Line22 – Line26). This is for displaying the View defined in CreateOrderItem.cshtml.
Then we create the [HttpPost], which needs special care, because if the users input some ItemIds or OrderIds that do not exist in the database, the OrderItem cannot be created successfully.
Thus, we write Line33 – Line34 to find out whether the user input ItemIds or OrderIds exist. If one of them cannot be found (namely null), a bad request is returned (Line36 – Line39). Otherwise, a new OrderItem is created, its price is calculated, and then added to the database (Line41 – Line52).

OrderController.cs

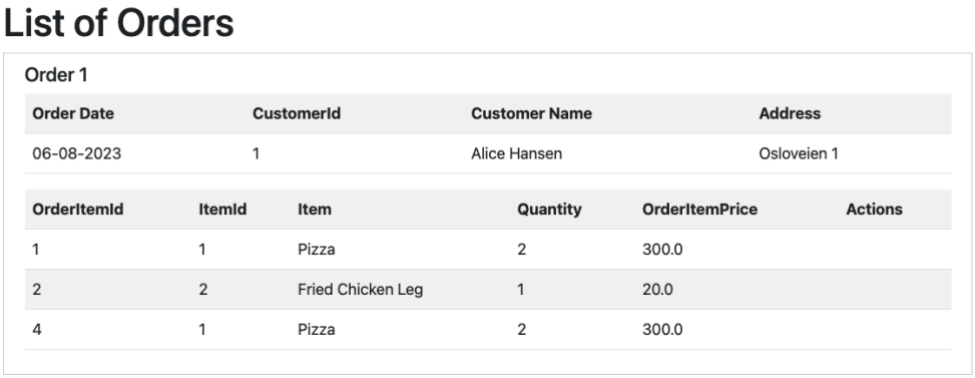
election

```
1   using Microsoft.AspNetCore.Mvc;
2   using MyShop.Models;
3   using Microsoft.EntityFrameworkCore;
4
5   namespace MyShop.Controllers;
6
7   public class OrderController : Controller
8   {
9       private readonly ItemDbContext _itemDbContext;
10
11      public OrderController(ItemDbContext itemDbContext)...
15
16      public async Task<IActionResult> Table()...
21
22      [HttpGet]
23      public IActionResult CreateOrderItem()
24      {
25          return View();
26      }
27
28      [HttpPost]
29      public async Task<IActionResult> CreateOrderItem(OrderItem orderItem)
30      {
31          try
32          {
33              var newItem = _itemDbContext.Items.Find(orderItem.ItemId);
34              var newOrder = _itemDbContext.Orders.Find(orderItem.OrderId);
35
36              if (newItem == null || newOrder == null)
37              {
38                  return BadRequest("Item or Order not found.");
39              }
40
41              var newOrderItem = new OrderItem
42              {
43                  ItemId = orderItem.ItemId,
44                  Item = newItem,
45                  Quantity = orderItem.Quantity,
46                  OrderId = orderItem.OrderId,
47                  Order = newOrder,
48              };
49              newOrderItem.OrderItemPrice = orderItem.Quantity * newOrderItem.Item.Price;
50
51              _itemDbContext.OrderItems.Add(newOrderItem);
52              await _itemDbContext.SaveChangesAsync();
53              return RedirectToAction(nameof(Table));
54          }
55          catch
56          {
57              return BadRequest("OrderItem creation failed.");
58          }
59      }
60  }
```

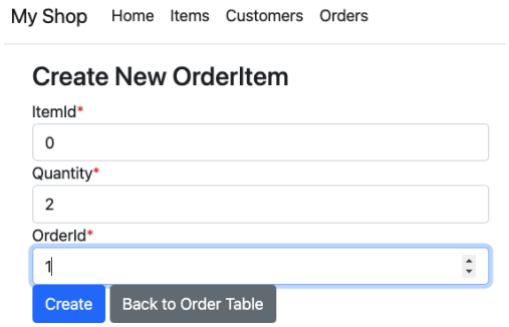
Now run the project and go to Create OrderItem, we input the following information:



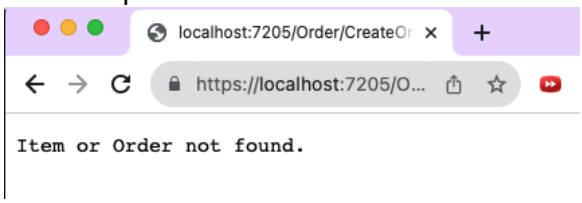
It can be seen that an OrderItem is successfully created.



If we try to create something that does not exist:



A bad request is returned:



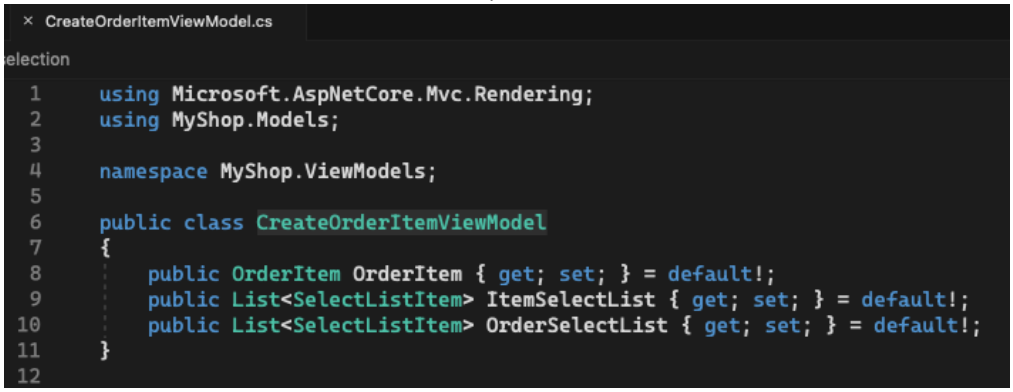
Dropdown List

We see the app makes it easy to make mistakes because the users may not remember what items exist. We can think about replace the forms for ItemIds and OrderIds as dropdown lists, and allow the users to select only from the existing items and orders.

To do so, we need to pass the list of existing Items and Orders to the CreateOrderItem.cshtml. For this, we need a View Model for creating OrderItems.

We add a new class named as CreateOrderItemViewModel.cs under ViewModels/:

The ViewModel contains a OrderItem, and two lists of SelectListItem of Items and Orders.



We then modify the Controllers/OrderController.cs as such:
Line5 imports a new dependency, which provides classes and utilities designed for rendering HTML elements in Razor views, including dropdown lists (also known as SelectLists).
Line24 – Line45 is the async way of [HttpGet]CreateOrderItem()
Line27 – Line28 retrieves the Items and Orders from the database and convert them to lists.
Line29 creates a new CreateOrderItemViewModel, which has three member variables: OrderItem, ItemSelectList, and OrderSelectList.
Line31 creates an empty OrderItem.
Line33 – Line37 wraps the information of ItemIds and Item Names into the ItemSelectList. Note that each SelectListItem has a value that is the content of the SelectListItem and a text that displays the time.
Line39 – Line43 wraps the information of OrderIds, OrderDates, and Customer Names into the OrderSelectList.

OrderController.cs

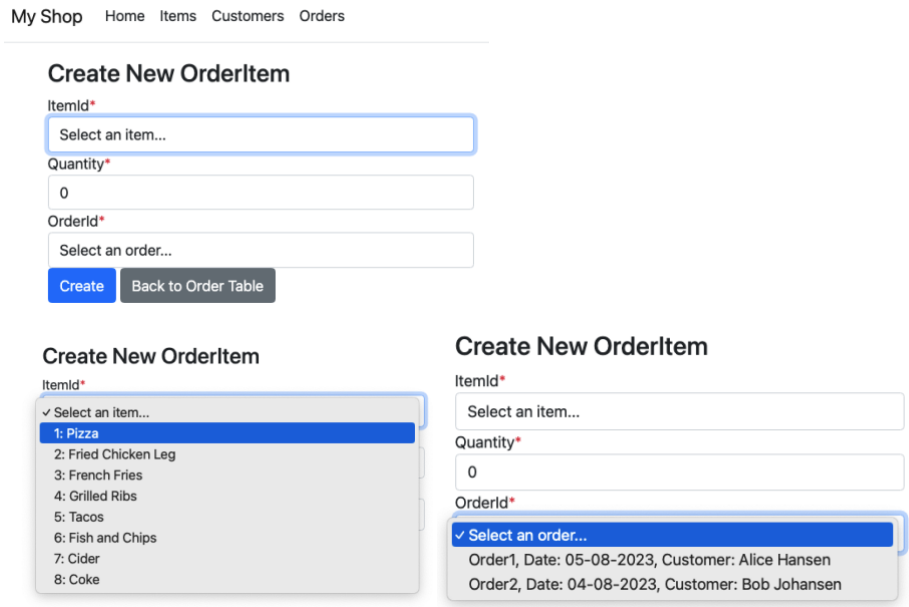
```
election
 1    using Microsoft.AspNetCore.Mvc;
 2    using MyShop.Models;
 3    using Microsoft.EntityFrameworkCore;
 4    using MyShop.ViewModels;
 5    using Microsoft.AspNetCore.Mvc.Rendering;
 6
 7    namespace MyShop.Controllers;
 8
 9    public class OrderController : Controller
10    {
11        private readonly ItemDbContext _itemDbContext;
12
13        public OrderController(ItemDbContext itemDbContext)...
17
18        public async Task<IActionResult> Table()...
23
24        [HttpGet]
25        public async Task<IActionResult> CreateOrderItem()
26        {
27            var items = await _itemDbContext.Items.ToListAsync();
28            var orders = await _itemDbContext.Orders.ToListAsync();
29            var createOrderItemViewModel = new CreateOrderItemViewModel
30            {
31                OrderItem = new OrderItem(),
32
33                ItemSelectList = items.Select(item => new SelectListItem
34                {
35                    Value = item.ItemId.ToString(),
36                    Text = item.ItemId.ToString() + ": " + item.Name
37                }).ToList(),
38
39                OrderSelectList = orders.Select(order => new SelectListItem
40                {
41                    Value = order.OrderId.ToString(),
42                    Text = "Order" + order.OrderId.ToString() + ", Date: " + order.OrderDate + ",
                      Customer: " + order.Customer.Name
43                }).ToList(),
44            };
45            return View(createOrderItemViewModel);
46        }
47
48        [HttpPost]
49        public async Task<IActionResult> CreateOrderItem(OrderItem orderItem)...
80    }
```

Now we need to modify the Views/Order/CreateOrderItem.cshtml to add the two dropdown lists:
Line1: modify the new ViewModel
Line6 – Line24, modify the asp-form to update the model binding, now we need to put OrderItem before the properties because the new OrderItem is a member variable of ViewModel, and the properties are member variables of the OrderItem.
Line8 – Line10: the select tag helper generate a dropdown list with the content of ItemSelectList.
Similarly, in Line20 – Line22, the select tag helper generate a dropdown list with the conent of OrderSelectList.

CreateOrderItem.cshtml

```
1   @model CreateOrderItemViewModel
2
3   <h2>Create New OrderItem</h2>
4
5   <form asp-action="CreateOrderItem">
6       <div class="form-group">
7           <label asp-for="OrderItem.ItemId"></label><span class="text-danger">*</span>
8           <select asp-for="OrderItem.ItemId" asp-items="Model.ItemSelectList" class="form-control">
9               <option value="">Select an item...</option>
10          </select>
11          <span asp-validation-for="OrderItem.ItemId" class="text-danger"></span>
12      </div>
13      <div class="form-group">
14          <label asp-for="OrderItem.Quantity"></label><span class="text-danger">*</span>
15          <input asp-for="OrderItem.Quantity" class="form-control" />
16          <span asp-validation-for="OrderItem.Quantity" class="text-danger"></span>
17      </div>
18      <div class="form-group">
19          <label asp-for="OrderItem.OrderId"></label><span class="text-danger">*</span>
20          <select asp-for="OrderItem.OrderId" asp-items="Model.OrderSelectList" class="form-control">
21              <option value="">Select an order...</option>
22          </select>
23          <span asp-validation-for="OrderItem.OrderId" class="text-danger"></span>
24      </div>
25      <button type="submit" class="btn btn-primary">Create</button>
26      <a asp-action="Table" class="btn btn-secondary">Back to Order Table</a>
27  </form>
```

Now start the project or using debugging (press F5) we can select from the item lists and order lists:



Great work! Now you have created more advanced functionality for CRUD. ☺
The rest of the CRUD for the expanded database is left for yourself to explore. ☺